Last updated in July 2026

Nyasha Hama

Backend & Platform Engineer | Go, TypeScript, Rust | Healthcare Ops, Data Systems & Upstream OSS

Cape Town, South Africa | nyashaahama@gmail.com | portfolio-topaz-one-58.vercel.app
github.com/nyashahama | linkedin.com/in/nyasha-hama-5b1312229

Summary

Backend and platform engineer building public, working systems in Go, TypeScript, Rust, PostgreSQL, and Redis, with emphasis on healthcare operations, property platforms, data workflows, auth/RBAC, observability, and release gates.

Recent evidence includes merged upstream Turso database-engine fixes, a merged CrossHair Python/C tracer contribution, a live clinic-operations platform for South Africa's public healthcare network, and a live sectional-title property platform with real workflow depth.

Skills

Languages: Go, TypeScript, Rust, SQL, JavaScript/Node.js, Java, C++

Backend, Data & Platform: REST APIs, service design, JWT auth, RBAC, PostgreSQL, Redis, pgx, sqlc, migrations, indexing, SQLite internals

Infrastructure & Quality: Docker, Linux, Prometheus, structured logging, request tracing, GitHub Actions, CI/CD, E2E tests, smoke/load checks, release gates

Experience

Backend & Platform Engineer, Independent Software Engineer – Remote Jan 2023 – present

- Contributed two merged Rust fixes to Turso's production SQLite-compatible database engine; preserved AUTOINCREMENT invariants after DROP COLUMN, cleared stale sqlite_sequence metadata after ALTER COLUMN, and patched schema serialization, ALTER execution state, and file-backed reopen coverage. Merged a Python/C tracer contribution to CrossHair moving call-target normalization into the C extension with full coverage.
- Built and shipped three live production-oriented systems: ClinicPulse covering 3,500+ public primary healthcare facilities across South Africa, StrataHQ for sectional-title property management (levy reconciliation, maintenance, AGM governance, WhatsApp, Stripe), and Healthcare Access Connector with HIPAA/POPIA-aligned patient-provider scheduling and telemedicine.
- Designed Go REST APIs with JWT/RBAC across multi-role hierarchies (admin, district, facility, field, partner, public), PostgreSQL schemas modeled with pgx, sqlc, and goose migrations, Redis-backed caching and rate limiting, NATS-based async messaging, and explicit SQL-first access patterns avoiding ORM abstractions.
- Added Prometheus metrics, structured logging, health/readiness endpoints, request tracing, graceful shutdown, Docker Compose multi-service environments, and GitHub Actions CI pipelines with race detection, migration verification, vulnerability scanning, and release-readiness gates across every project.
- Created a reusable Go backend scaffold with documented quality gates, adoption checklist, bootstrap verification, and release automation (binary builds and Docker images via GitHub Releases/GHCR), designed so an adopter can clone, verify, and ship within hours.
- Building a Rust policy-enforcement runtime (guard-rail) for internal API traffic with route-level authorization, audit persistence, replay capture, readiness and metrics surfaces, and a documented beta deployment path covering container, Postgres, and reverse-proxy infrastructure.
- Sustained independent engineering across 3+ years, shipping systems with 800+ commits each, managing scope across Go, TypeScript, Rust, PostgreSQL, Redis, Docker, and CI/CD without daily supervision or a supporting team.

Open Source Contributions

Upstream Contributor, Turso May 2026

- Merged two Rust database-engine fixes around SQLite-compatible schema rewrites, preserving AUTOINCREMENT after DROP COLUMN and clearing stale sqlite_sequence metadata after ALTER COLUMN removes an AUTOINCREMENT rowid alias.
- Patched schema serialization, ALTER execution state, SQL regressions, and file-backed reopen integration coverage for rowid reuse and stale metadata failures.
- Links: PR #6993 | PR #7117

Upstream Contributor, CrossHair May 2026

- Merged Python/C tracer work moving call target normalization into CrossHair's C extension while keeping keyword handling and trace dispatch behavior stable.
- Added coverage for bound Python methods, callable instances, C-level bound methods, reference behavior, and descriptor-error propagation.
- Links: PR #413

Projects

ClinicPulse

Live TypeScript and Go clinic-operations platform for South Africa's public primary healthcare network, covering facility status, referral routing, field reporting, and district operational intelligence.

- Built role-based workflows for admin, district, facility, field, partner, and public users, with PostgreSQL-backed API paths and product surfaces for operational decision-making.
- Added release gates, auth/security hardening, pilot data integrity, partner workflows, E2E coverage, smoke/load scripts, and observability primitives including request correlation, metrics, and runbooks.
- Links: GitHub | Live

StrataHQ

Live Next.js and Go platform for South African sectional-title property operations, serving agents, trustees, and residents across scheme management workflows.

- Built scheme-scoped levy operations, maintenance workflows, resident/trustee communications, AGM administration, document access, audit logs, open API access, and predictive levy analytics.
- Modeled production-style backend paths with PostgreSQL, pgx, sqlc, goose migrations, Redis, auth/RBAC, tests/CI, and deployment-ready project structure.
- Links: GitHub | Live

Healthcare Access Connector

Go healthcare access backend connecting patients and providers through scheduling, search, notifications, telemedicine support, and role-aware workflows.

- Designed patient, provider, appointment, staff invitation, and notification flows with PostgreSQL-backed domain modeling, validation, and explicit service boundaries.
- Implemented JWT/RBAC, rate limiting, Redis caching, NATS messaging, Docker local infrastructure, Prometheus metrics, structured logging, health endpoints, and graceful shutdown.
- Links: GitHub

Education

University of Johannesburg — BSc Computer Science Coursework, completed semesters through final-year coursework, 2022–2024